New Horizons
Centro de Capacitación

Práctica: Servicio HTTP, Environment y Conexión CRUD con el Backend C#

Paso a paso — Configurar provideHttpClient, environments y SolicitudHttpService conectando a los endpoints de ASP.NET Core

Curso	Técnico de Desarrollo Full Stack con C# y Angular
Sesión	Sesión 11 — Consumo de la API REST de C# con HttpClient y RxJS
Docente	Edwin Barrientos
Duración estimada	60–90 minutos
Nivel	Básico (guía principal de la sesión)
Entorno	.NET 8 SDK · Visual Studio 2022 Community o VS Code · PostgreSQL o SQL Server Express

1. Objetivos

- Configurar provideHttpClient(withFetch()) en app.config.ts.
- Definir variables de entorno en environment.ts y environment.development.ts apuntando a la API en localhost:5000.
- Construir SolicitudHttpService implementando peticiones GET, POST, PUT y DELETE tipadas fuertemente.
- Conectar la MatTable y el formulario reactivo a los endpoints de la API de C# desarrollada previamente.
- Verificar la persistencia real de datos en base de datos mediante pruebas de integración.

2. Desarrollo paso a paso

Paso 1 — Configurar environments con la URL del Backend C#

Creamos los archivos de entorno en src/environments/environment.ts y src/environments/environment.development.ts:

`src/environments/environment.ts`

```
export const environment = {
  production: false,
  apiUrl: 'http://localhost:5000/api'
};
```

src/environments/environment.development.ts

```
export const environment = {
  production: false,
  apiUrl: 'http://localhost:5000/api'
};
```

Paso 2 — Habilitar HttpClient en app.config.ts

Configuramos el cliente HTTP nativo con la API moderna provideHttpClient en src/app/app.config.ts:

src/app/app.config.ts

```
import { ApplicationConfig, provideZoneChangeDetection } from '@angular/core';
import { provideRouter } from '@angular/router';
import { provideHttpClient, withFetch, withInterceptors } from '@angular/common/http';
import { provideAnimationsAsync } from '@angular/platform-browser/animations/async';
import { routes } from './app.routes';
import { loadingInterceptor } from './interceptors/loading.interceptor';
import { errorInterceptor } from './interceptors/error.interceptor';

export const appConfig: ApplicationConfig = {
  providers: [
    provideZoneChangeDetection({ eventCoalescing: true }),
    provideRouter(routes),
    provideHttpClient(
      withFetch(),
      withInterceptors([loadingInterceptor, errorInterceptor])
    ),
    provideAnimationsAsync()
  ]
};
```

Paso 3 — Implementar SolicitudHttpService con Tipado Estricto

Construimos el servicio HTTP que consume los endpoints reales del backend en src/app/services/solicitud-http.service.ts:

src/app/services/solicitud-http.service.ts

```
import { Injectable, inject, signal } from '@angular/core';
import { HttpClient, HttpParams } from '@angular/common/http';
import { Observable, tap } from 'rxjs';
import { environment } from '../../environments/environment';
import { Solicitud, PagedResult, CrearSolicitudDto, ActualizarSolicitudDto, TimelineItem,
EstadoSolicitud } from '../models/solicitud.model';

@Injectable({ providedIn: 'root' })
export class SolicitudHttpService {
  private http = inject(HttpClient);
  private baseUrl = `${environment.apiUrl}/solicitudes`;

  solicitudes = signal<Solicitud[]>([]);

  obtenerPaginado(pagina: number = 1, tamano: number = 10, estado?: EstadoSolicitud):
Observable<PagedResult<Solicitud>> {
    let params = new HttpParams().set('pagina', pagina.toString()).set('tamanoPagina',
tamano.toString());
    if (estado !== undefined && estado !== null) {
      params = params.set('estado', estado.toString());
    }
    return this.http.get<PagedResult<Solicitud>>(this.baseUrl, { params }).pipe(
      tap(res => this.solicitudes.set(res.items))
    );
  }

  obtenerPorId(id: number): Observable<Solicitud> {
    return this.http.get<Solicitud>(`${this.baseUrl}/${id}`);
  }

  crear(dto: CrearSolicitudDto): Observable<Solicitud> {
    return this.http.post<Solicitud>(this.baseUrl, dto).pipe(
      tap(nueva => this.solicitudes.update(list => [nueva, ...list]))
    );
  }

  actualizar(id: number, dto: ActualizarSolicitudDto): Observable<void> {
    return this.http.put<void>(`${this.baseUrl}/${id}`, dto);
  }

  eliminar(id: number): Observable<void> {
    return this.http.delete<void>(`${this.baseUrl}/${id}`).pipe(
      tap(() => this.solicitudes.update(list => list.filter(s => s.id !== id)))
    );
  }

  obtenerTimeline(id: number): Observable<TimelineItem[]> {
    return this.http.get<TimelineItem[]>(`${this.baseUrl}/${id}/timeline`);
  }
}
```

Paso 4 — Conectar SolicitudesListComponent, SolicitudFormComponent y SolicitudDetailComponent a la API Real de C#

Actualizamos los componentes de páginas (con sus 3 archivos separados .ts, .html y .css) para que consuman SolicitudHttpService:

`src/app/pages/solicitudes-list/solicitudes-list.component.ts`

```
import { Component, OnInit, ViewChild, inject, effect } from '@angular/core';
import { CommonModule } from '@angular/common';
import { RouterLink } from '@angular/router';
import { MatTableDataSource, MatTableModule } from '@angular/material/table';
import { MatSort, MatSortModule } from '@angular/material/sort';
import { MatPaginator, MatPaginatorIntl, MatPaginatorModule } from '@angular/material/paginator';
import { MatFormFieldModule } from '@angular/material/form-field';
import { MatInputModule } from '@angular/material/input';
import { MatChipsModule } from '@angular/material/chips';
import { MatIconModule } from '@angular/material/icon';
import { MatButtonModule } from '@angular/material/button';
import { MatDialog, MatDialogModule } from '@angular/material/dialog';
import { MatSnackBar } from '@angular/material/snack-bar';
import { SolicitudHttpService } from '../../services/solicitud-http.service';
import { Solicitud, EstadoSolicitud } from '../../models/solicitud.model';
import { MatPaginatorIntlEs } from '../../shared/mat-paginator-es';
import { ConfirmDialogComponent } from '../../components/confirm-dialog/confirm-
dialog.component';

@Component({
  selector: 'app-solicitudes-list',
  standalone: true,
  providers: [{ provide: MatPaginatorIntl, useClass: MatPaginatorIntlEs }],
  imports: [
    CommonModule, RouterLink, MatTableModule, MatSortModule, MatPaginatorModule,
    MatFormFieldModule, MatInputModule, MatChipsModule, MatIconModule, MatButtonModule,
MatDialogModule
  ],
  templateUrl: './solicitudes-list.component.html',
  styleUrl: './solicitudes-list.component.css'
})
export class SolicitudesListComponent implements OnInit {
  private httpService = inject(SolicitudHttpService);
  private dialog = inject(MatDialog);
  private snackBar = inject(MatSnackBar);

  columnasVisibles: string[] = ['codigo', 'motivo', 'solicitante', 'montoTotal', 'estado',
'acciones'];
  dataSource = new MatTableDataSource<Solicitud>([]);

  @ViewChild(MatSort) sort!: MatSort;
  @ViewChild(MatPaginator) paginator!: MatPaginator;

  constructor() {
    effect(() => { this.dataSource.data = this.httpService.solicitudes(); });
  }

  ngOnInit(): void {
    this.cargarDatos();
  }

  ngAfterViewInit(): void {
    this.dataSource.sort = this.sort;
    this.dataSource.paginator = this.paginator;
  }

  cargarDatos(): void {
    this.httpService.obtenerPaginado(1, 50).subscribe({
      next: (res) => { this.dataSource.data = res.items; },
      error: () => {
        // En caso de que el backend no esté levantado, usa datos locales
        this.dataSource.data = this.httpService.solicitudes();
      }
    });
  }

  aplicarFiltro(event: Event): void {
    const valor = (event.target as HTMLInputElement).value;
    this.dataSource.filter = valor.trim().toLowerCase();
  }

  obtenerClaseEstado(estado: EstadoSolicitud): string {
    switch (estado) {
      case EstadoSolicitud.Borrador: return 'chip-borrador';
      case EstadoSolicitud.Enviada: return 'chip-enviada';
      case EstadoSolicitud.EnRevision: return 'chip-enrevision';
      case EstadoSolicitud.Aprobada: return 'chip-aprobada';
```

New Horizons
Centro de Capacitación

`src/app/pages/solicitud-form/solicitud-form.component.ts`

```
import { Component, OnInit, inject } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormBuilder, FormGroup, FormArray, ReactiveFormsModule, Validators } from
'@angular/forms';
import { Router, RouterLink } from '@angular/router';
import { MatFormFieldModule } from '@angular/material/form-field';
import { MatInputModule } from '@angular/material/input';
import { MatSelectModule } from '@angular/material/select';
import { MatButtonModule } from '@angular/material/button';
import { MatCardModule } from '@angular/material/card';
import { MatIconModule } from '@angular/material/icon';
import { MatSnackBar } from '@angular/material/snack-bar';
import { SolicitudHttpService } from '../../services/solicitud-http.service';
import { motivoMinimoPalabrasValidator } from '../../validators/custom-validators';

@Component({
  selector: 'app-solicitud-form',
  standalone: true,
  imports: [
    CommonModule, ReactiveFormsModule, RouterLink,
    MatFormFieldModule, MatInputModule, MatSelectModule, MatButtonModule, MatCardModule,
MatIconModule
  ],
  templateUrl: './solicitud-form.component.html',
  styleUrl: './solicitud-form.component.css'
})
export class SolicitudFormComponent implements OnInit {
  private fb = inject(FormBuilder);
  private router = inject(Router);
  private httpService = inject(SolicitudHttpService);
  private snackBar = inject(MatSnackBar);

  formulario!: FormGroup;

  centrosCosto = [
    { id: 1, codigo: 'CC-001', nombre: 'Tecnologías de la Información' },
    { id: 2, codigo: 'CC-002', nombre: 'Operaciones y Logística' },
    { id: 3, codigo: 'CC-003', nombre: 'Ventas y Marketing' },
    { id: 4, codigo: 'CC-004', nombre: 'Administración y Finanzas' }
  ];

  ngOnInit(): void {
    this.formulario = this.fb.group({
      motivo: ['', [Validators.required, Validators.minLength(10),
motivoMinimoPalabrasValidator(3)]],
      centroCostoId: [null, [Validators.required]],
      solicitanteId: [1],
      detalles: this.fb.array([], [Validators.required])
    });

    this.agregarFila();
  }

  get detalles(): FormArray {
    return this.formulario.get('detalles') as FormArray;
  }

  crearFilaDetalle(): FormGroup {
    return this.fb.group({
      concepto: ['', [Validators.required, Validators.minLength(3)]],
      cantidad: [1, [Validators.required, Validators.min(1)]],
      precioUnitario: [0, [Validators.required, Validators.min(0.1)]]
    });
  }

  agregarFila(): void {
    this.detalles.push(this.crearFilaDetalle());
  }

  eliminarFila(index: number): void {
    if (this.detalles.length > 1) {
      this.detalles.removeAt(index);
    }
  }

  calcularSubtotal(index: number): number {
    const fila = this.detalles.at(index);
```

src/app/pages/solicitud-detail/solicitud-detail.component.ts

```
import { Component, OnInit, inject } from '@angular/core';
import { CommonModule } from '@angular/common';
import { ActivatedRoute, RouterLink } from '@angular/router';
import { MatCardModule } from '@angular/material/card';
import { MatButtonModule } from '@angular/material/button';
import { MatIconModule } from '@angular/material/icon';
import { MatTableModule } from '@angular/material/table';
import { SolicitudHttpService } from '../../services/solicitud-http.service';
import { Solicitud } from '../../models/solicitud.model';

@Component({
  selector: 'app-solicitud-detail',
  standalone: true,
  imports: [CommonModule, RouterLink, MatCardModule, MatButtonModule, MatIconModule,
MatTableModule],
  templateUrl: './solicitud-detail.component.html',
  styleUrl: './solicitud-detail.component.css'
})
export class SolicitudDetailComponent implements OnInit {
  private route = inject(ActivatedRoute);
  private httpService = inject(SolicitudHttpService);
  solicitud?: Solicitud;

  ngOnInit(): void {
    const id = Number(this.route.snapshot.paramMap.get('id'));
    this.httpService.obtenerPorId(id).subscribe({
      next: (data) => { this.solicitud = data; }
    });
  }
}
```

3. Retos adicionales

1. Configura un proxy de desarrollo en proxy.conf.json para redirigir peticiones /api a localhost:5000 evitando problemas locales de CORS.
2. Implementa paginación remota conectando los eventos (page) de MatPaginator al método obtenerPaginado(pagina, tamano).
3. Agrega un botón de recarga (refresh) que vuelva a consultar el backend mostrando una animación giratoria.
4. Implementa debounceTime con RxJS en el campo de búsqueda para consultar la API solo cuando el usuario deje de escribir.

4. Entregables

- Conexión exitosa verificada entre solicitudes-frontend y SolicitudesApp.Api de C#.
- GET /api/solicitudes respondiendo y cargando datos reales en la tabla de Material.
- POST /api/solicitudes persistiendo una nueva solicitud en base de datos desde el formulario.
- Eliminación física o lógica con DELETE /api/solicitudes/{id} funcionando.